

Practical: 7

Implementation of making a change problem using dynamic programming

15/09/2026

Aim: To implement of making a change problem using dynamic programming

Algorithm:

Algorithm. Practical 7.

Step 1 : Start

Step 2 : store the available coin denominations

Step 3 : read the required amount A .

Step 4 : create a DP array $dp[0, \dots, A]$

Step 5 : ~~Set~~ Set

- i) $dp[0] = 0$
- ii) All other values = ∞

Step 6 : for every amount i from 1 to A ;

- i) check every coin denomination
- ii) if $coin \leq i$, calculate
 $dp[i] = \min(dp[i], 1 + dp[i - coin])$

Step 7 : After filling the DP table $dp[A]$ contains the minimum no. of coins

Step 8 : print the min no. of coins

Step 9 : stop.

Program:

```
#include <iostream>
#include <algorithm>
using namespace std;

int main()
{
    // Static coin denominations
    int coins[] = {1, 2, 5, 10};

    int n = 4;
    int amount = 12;

    int dp[100];

    // Base case
    dp[0] = 0;

    // Initialize DP array
    for (int i = 1; i <= amount; i++)
        dp[i] = 9999;

    // Dynamic Programming
    for (int i = 1; i <= amount; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (coins[j] <= i)
            {
                dp[i] = min(dp[i],
                    1 + dp[i - coins[j]]);
            }
        }
    }

    cout << "Coins: 1 2 5 10" << endl;
    cout << "Amount: " << amount << endl;

    cout << "Minimum number of coins = "
        << dp[amount] << endl;

    return 0;
}
```

Output:

Output

```
Coins: 1 2 5 10
Amount: 12
Minimum number of coins = 2
```

Time Complexity:

Time Complexity:

A = amount
 n = no. of coins : denominations
 $\therefore T(A, n) = A \times n$
 $\therefore$ Time complexity: $O(n \cdot A)$
 $\therefore O(n \cdot A)$
 $\therefore$ Space complexity: $O(A)$ Due to the $A+1$

Best case: $O(n \cdot A)$
Average case: $O(n \cdot A)$
Worst case: $O(n \cdot A)$

Conclusion:

The Implementation of making a change problem using dynamic programming is successfully executed with the time complexity of $O(nA)$ along with same for all the three cases.

Best case = Average case = Worst case = $O(nA)$.